

Nesneye Yönelik Programlama

Hafta 5

Örnek 1

Düzenli Çokgen (Regular Polygon): n kenarlı bir düzgün çokgende (regular polygon), tüm kenarlar eşit uzunluktadır ve tüm açılar eşittir (yani çokgen hem eşkenar hem de eşaçılıdır). Aşağıdaki özelliklere sahip **RegularPolygon** adında bir sınıf tasarlayınız.

Veri Alanları (Private):

- **n** adında bir **int** değişken → çokgenin kenar sayısını tutar (varsayılan değer: 3)
- **side** adında bir **double** değişken → kenar uzunluğunu tutar (varsayılan değer: 1)
- **x** adında bir **double** değişken → çokgenin merkezinin x koordinatı (varsayılan değer: 0)
- **y** adında bir **double** değişken → çokgenin merkezinin y koordinatı (varsayılan değer: 0)

Constructor'lar:

- Parametresiz (no-arg) constructor → varsayılan değerlerle nesne oluşturur.
- **(n, side)** parametrelili constructor → merkez (0,0) olacak şekilde nesne oluşturur.
- **(n, side, x, y)** parametrelili constructor → tüm değerleri verilen nesne oluşturur.

Örnek 1

Metotlar:

- Tüm veri alanları için **getter (accessor)** ve **setter (mutator)** metotları
- **getPerimeter()** → çokgenin çevresini hesaplayıp döndürür.
- **getArea()** → çokgenin

$$Alan = \frac{n \times side^2}{4 \times \tan(\pi/n)}$$

Alan hesaplama formülü:

İstenenler:

1. **RegularPolygon** sınıfını gerçekleştiriniz (implement ediniz).
2. Aşağıdaki şekilde üç adet **RegularPolygon** nesnesi oluşturan bir test programı yazınız:
 - Parametresiz constructor ile
 - **RegularPolygon(6, 4)** kullanarak
 - **RegularPolygon(10, 4, 5.6, 7.8)** kullanarak
3. Her bir nesne için çevre ve alan değerlerini ekrana yazdırınız.

```
package btu_lab;
```

```
public class RegularPolygon {
```

```
    // private data fields  
    private int n = 3;  
    private double side = 1;  
    private double x = 0;  
    private double y = 0;
```

```
    // no-arg constructor  
    public RegularPolygon() {  
    }
```

```
    // constructor (n, side)  
    public RegularPolygon(int n, double side) {  
        this.n = n;  
        this.side = side;  
    }
```

```
    // constructor (n, side, x, y)  
    public RegularPolygon(int n, double side, double x, double y) {  
        this.n = n;  
        this.side = side;  
        this.x = x;  
        this.y = y;  
    }
```

```
    public void setN(int n) {  
        this.n = n;  
    }
```

```
    public int getN() {  
        return n;  
    }
```

```
    public void setSide(double side) {  
        this.side = side;  
    }
```

```
    public double getSide() {  
        return side;  
    }
```

```
    public void setX(double x) {  
        this.x = x;  
    }
```

```
    public double getX() {  
        return x;  
    }
```

```
    public void setY(double y) {  
        this.y = y;  
    }
```

```
    public double getY() {  
        return y;  
    }
```

```
    // perimeter  
    public double getPerimeter() {  
        return n * side;  
    }
```

```
    // area  
    public double getArea() {  
        return (n * side * side) / (4 * Math.tan(Math.PI / n));  
    }
```

```
}
```

Örnek 2

- **(Cebir: 2×2 Lineer Denklemler):** Aşağıdaki 2×2 lineer denklem sistemini temsil eden LinearEquation a $ax + by = e$ mız:

$$cx + dy = f$$

-

$$x = \frac{ed - bf}{ad - bc} \quad y = \frac{af - ec}{ad - bc}$$

Sınıfın İçermesi Gerekenler:

- a, b, c, d, e, f adında **private veri alanları**
- Bu değişkenleri parametre olarak alan bir constructor
- a, b, c, d, e, f için getter metotları
- `isSolvable()` $\rightarrow ad - bc \neq 0$ ise true döndürür.
- `getX()` ve `getY()` $\rightarrow$ çözümü döndürür.

İstenenler:

1. Sınıfı implemente ediniz.
2. Kullanıcıdan a, b, c, d, e, f değerlerini alan bir test programı yazınız.
3. Eğer $ad - bc = 0$ ise:

"The equation has no solution." yazdırınız.

1. Aksi halde x ve y değerlerini yazdırınız.

```

package btu;

public class LinearEquation {

    // private data fields
    private double a, b, c, d, e, f;

    // constructor
    public LinearEquation(double a, double b, double c, double d, double e, double f) {
        this.a = a;
        this.b = b;
        this.c = c;
        this.d = d;
        this.e = e;
        this.f = f;
    }

    // getters
    public double getA() { return a; }
    public double getB() { return b; }
    public double getC() { return c; }
    public double getD() { return d; }
    public double getE() { return e; }
    public double getF() { return f; }

    // solvable mı?
    public boolean isSolvable() {
        return (a * d - b * c) != 0;
    }

    // x değeri
    public double getX() {
        return (e * d - b * f) / (a * d - b * c);
    }

    // y değeri
    public double getY() {
        return (a * f - e * c) / (a * d - b * c);
    }
}

```

```

package btu;
import java.util.Scanner;

public class Test {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("a, b, c, d, e, f değerlerini girin: ");
        double a = input.nextDouble();
        double b = input.nextDouble();
        double c = input.nextDouble();
        double d = input.nextDouble();
        double e = input.nextDouble();
        double f = input.nextDouble();

        LinearEquation eq = new LinearEquation(a, b, c, d, e, f);

        if (eq.isSolvable()) {
            System.out.println("x = " + eq.getX());
            System.out.println("y = " + eq.getY());
        } else {
            System.out.println("The equation has no solution.");
        }
    }
}

```